Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №7
Варіант 14
На тему
«functools та operator»

Виконав:
Студент групи ФЕП-21с
Хороший Святослав

Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи

Застосувати partial для налаштування функцій, reduce для побудови конвеєра та агрегації, singledispatch для поліморфного завантаження даних і total_ordering для повного порядку. Використати засоби operator для полів, атрибутів та викликів методів.

Теоретичні відомості

Partial фіксує частину аргументів функції. Reduce послідовно згортає значення з початковим акумулятором; у pipeline ним є поточні дані. Itemgetter дістає поля, attrgetter — атрибути, methodcaller викликає метод об’єкта.

Singledispatch вибирає реалізацію за типом першого аргументу. Total_ordering добудовує оператори порівняння на основі __eq__ і __lt__. Для чужого типу потрібно повертати NotImplemented.

Хід роботи

Для варіанта 14 я реалізував логістичні відправлення з полями id, weight, distance, priority та cost. Нормалізація перетворює числові поля та очищає priority через strip і lower. Вихідна ShipmentRank є незмінною моделлю.

Для прикладу обрав формулу adjusted_cost = (cost + weight × 2 + distance × 0,1) × 1,2. Умова не задає ставок, тому тарифи передаються як параметри. Partial налаштовує коригування, фільтр, сортування та Top-N; reduce зв’язує етапи та окремо підсумовує відібрану вартість.

Конвеєр ліниво нормалізує записи, коригує вартість, залишає priority=high і сортує за вартістю спадно та id зростаюче. Для двох ключів використано послідовні стабільні sorted з attrgetter. Вхідні записи не змінюються.

У loaders.py я реалізував load_shipments на singledispatch для str, Path, відкритого текстового файлу та готового списку. Підтримуються JSON і CSV, включно зі StringIO. Кожний запис перевіряється, а предметні поля нормалізуються в core.py. Для list використано явну реєстрацію register(list).

ShipmentRank порівнюється за ключем (−adjusted_cost, id), тому sorted об’єктів відповідає рейтингу. У тестах перевірено всі шість операторів і поведінку з чужими типами. Еквівалентність форматів перевірено за всіма полями через asdict, а також підтверджено незмінність джерела.

Main.py запускає демонстрацію з sample_data.json і sample_data.csv. Очікуваний Top-3: №1, №3, №4 із вартістю 84,00; 84,00; 34,80. Загальна скоригована вартість становить 202,80. Нижче наведено код та фактичний запуск.

lab07 / models.py

```
"""Типізований запис рейтингу відправлень."""

from dataclasses import dataclass
from functools import total_ordering
from math import isfinite


@total_ordering
@dataclass(frozen=True, slots=True, eq=False)
class ShipmentRank:
    id: int
    weight: float
    distance: float
    priority: str
    cost: float
    adjusted_cost: float

    def __post_init__(self) -> None:
        if (
            self.id < 0
            or self.weight <= 0
```

Рис. 1. Файл models.py, рядки 1–21

lab07 / models.py

```
            or self.priority not in {"high", "normal", "low"}
        ):
            raise ValueError("Некоректний id, weight або priority")
        if any(
            not isfinite(x) or x < 0
            for x in (self.weight, self.distance, self.cost, self.adjusted_cost)
        ):
            raise ValueError("Числові поля мають бути скінченними й невід'ємними")

    def rank_key(self) -> tuple[float, int]:
        return -self.adjusted_cost, self.id

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, ShipmentRank):
            return NotImplemented
        return self.rank_key() == other.rank_key()

    def __lt__(self, other: object) -> bool:
        if not isinstance(other, ShipmentRank):
            return NotImplemented
        return self.rank_key() < other.rank_key()
```

Рис. 2. Файл models.py, рядки 22–42

lab07 / loaders.py

```python
"""Поліморфне завантаження JSON/CSV. I/O ізольовано в цьому модулі."""

import csv
import io
import json
from functools import singledispatch
from pathlib import Path

Row = dict[str, object]


def validate_rows(data: object) -> list[Row]:
    if not isinstance(data, list):
        raise ValueError("Очікується список словників")
    result: list[Row] = []
    for row in data:
        if not isinstance(row, dict) or not all(isinstance(key, str) for key in row):
            raise ValueError("Кожний запис має бути словником із рядковими ключами")
        result.append(dict(row))
    return result


def parse_text(text: str, *, csv_format: bool = False) -> list[Row]:
    if csv_format:
        return validate_rows(list(csv.DictReader(io.StringIO(text), strict=True)))
    data: object = json.loads(text)
    return validate_rows(data)


@singledispatch
def load_shipments(source: object) -> list[Row]:
    raise TypeError(f"Непідтримуваний тип джерела: {type(source).__name__}")

```

Рис. 3. Файл loaders.py, рядки 1–33

lab07 / loaders.py

```python

@load_shipments.register
def from_string(source: str) -> list[Row]:
    stripped = source.strip()
    if stripped.startswith(("[", "{")):
        return parse_text(stripped)
    if "\n" in source:
        return parse_text(source, csv_format=True)
    return load_shipments(Path(source))


@load_shipments.register
def from_path(source: Path) -> list[Row]:
    if source.suffix.lower() not in {".json", ".csv"}:
        raise ValueError("Файл має мати розширення .json або .csv")
    return parse_text(
        source.read_text(encoding="utf-8"), csv_format=source.suffix.lower() == ".csv"
    )


@load_shipments.register
def from_file(source: io.TextIOBase) -> list[Row]:
    text = source.read()
    name = str(getattr(source, "name", ""))
    csv_format = name.lower().endswith(".csv") or (
        not name.lower().endswith(".json") and not text.lstrip().startswith(("[", "{"))
    )
    return parse_text(text, csv_format=csv_format)


@load_shipments.register(list)
def from_list(source: list[object]) -> list[Row]:
    return validate_rows(source)
```

Рис. 4. Файл loaders.py, рядки 34–66

lab07 / core.py

```python
"""Чистий конвеєр: normalize → adjust → filter → sort → Top-N."""

from collections.abc import Callable, Iterable, Mapping
from dataclasses import replace
from functools import partial, reduce
from itertools import islice
from math import isfinite
from operator import attrgetter, itemgetter, methodcaller
from typing import TypeVar

from models import ShipmentRank

T = TypeVar("T")


def pipeline(value: T, *funcs: Callable[[T], T]) -> T:
    return reduce(lambda current, fn: fn(current), funcs, value)


def normalize(row: Mapping[str, object]) -> ShipmentRank:
    try:
        identifier = int(str(itemgetter("id")(row)).strip())
        numbers = [
            float(str(itemgetter(key)(row)).strip())
            for key in ("weight", "distance", "cost")
        ]
        priority = str(methodcaller("strip")(str(itemgetter("priority")(row))))
        priority = str(methodcaller("lower")(priority))
        weight, distance, cost = numbers
```

Рис. 5. Файл core.py, рядки 1–29

lab07 / core.py

```python
        return ShipmentRank(identifier, weight, distance, priority, cost, cost)
    except (KeyError, TypeError, ValueError) as error:
        raise ValueError(f"Некоректний запис відправлення: {row!r}") from error


def adjust(
    shipment: ShipmentRank, *, per_kg: float, per_km: float, factor: float
) -> ShipmentRank:
    cost = (
        shipment.cost + shipment.weight * per_kg + shipment.distance * per_km
    ) * factor
    return replace(shipment, adjusted_cost=cost)


def has_priority(shipment: ShipmentRank, *, priority: str) -> bool:
    return shipment.priority == priority


def take_top(items: Iterable[ShipmentRank], *, n: int) -> list[ShipmentRank]:
    return list(islice(items, n))


def rank_shipments(
    rows: Iterable[Mapping[str, object]],
    *,
    top: int = 3,
    priority: str = "high",
    per_kg: float = 2.0,
    per_km: float = 0.1,
```

Рис. 6. Файл core.py, рядки 30–58

lab07 / core.py

```
    factor: float = 1.2,
) -> list[ShipmentRank]:
    if top < 0 or priority not in {"high", "normal", "low"}:
        raise ValueError("Некоректний top або priority")
    if any(not isfinite(x) or x < 0 for x in (per_kg, per_km, factor)):
        raise ValueError("Тарифи мають бути скінченними й невід'ємними")
    stages: tuple[Callable[[Iterable[ShipmentRank]], Iterable[ShipmentRank]], ...] = (
        partial(map, partial(adjust, per_kg=per_kg, per_km=per_km, factor=factor)),
        partial(filter, partial(has_priority, priority=priority)),
        partial(sorted, key=attrgetter("id")),
        partial(sorted, key=attrgetter("adjusted_cost"), reverse=True),
        partial(take_top, n=top),
    )
    normalized: Iterable[ShipmentRank] = map(normalize, rows)
    return list(pipeline(normalized, *stages))


def total_cost(shipments: Iterable[ShipmentRank]) -> float:
    return reduce(
        lambda total, shipment: total + shipment.adjusted_cost, shipments, 0.0
    )


def clean_and_rank(source: object, *, top: int = 3) -> list[ShipmentRank]:
    """Оболонка завантаження та чистого конвеєра; підтримує всі джерела loader."""
    from loaders import load_shipments

    return rank_shipments(load_shipments(source), top=top)
```

Рис. 7. Файл core.py, рядки 59–86

lab07 / main.py

```
"""Демонстрація завантажувача та рейтингу, варіант 14."""

from operator import attrgetter
from pathlib import Path

from core import rank_shipments, total_cost
from loaders import load_shipments


def main() -> None:
    folder = Path(__file__).parent
    json_rows = load_shipments(folder / "sample_data.json")
    csv_rows = load_shipments(folder / "sample_data.csv")
    ranked = rank_shipments(json_rows)
    print("=== ЛОГІСТИЧНІ ВІДПРАВЛЕННЯ | ВАРІАНТ 14 ===")
    print("Формула: (cost + weight * 2 + distance * 0.1) * 1.2")
    print("Пріоритет: high; Top-3")
    for shipment in ranked:
        print(
            f"#{shipment.id}: {shipment.weight:g} кг, {shipment.distance:g} км, вартість
{shipment.adjusted_cost:.2f}"
        )
    print(f"Загальна скоригована вартість: {total_cost(ranked):.2f}")
    print("JSON і CSV дають однаковий рейтинг:", ranked == rank_shipments(csv_rows))
    print("Сортування через total_ordering:", [s.id for s in sorted(ranked)])
    print("Атрибути id через attrgetter:", list(map(attrgetter("id"), ranked)))
    print(
        "Порівняння першого й другого:", ranked[0] < ranked[1], ranked[0] <= ranked[1]
    )


if __name__ == "__main__":
    main()
```

Рис. 8. Файл main.py, рядки 1–32

lab07 / tests/test_lab07.py

```python
import io
import json
from copy import deepcopy
from dataclasses import asdict
from functools import partial
from operator import attrgetter, itemgetter, methodcaller
from pathlib import Path

import pytest

from core import adjust, clean_and_rank, normalize, pipeline, rank_shipments, total_cost
from loaders import load_shipments


@pytest.fixture
def rows() -> list[dict[str, object]]:
    return [
        {"id": 3, "weight": 5, "distance": 100, "priority": " HIGH ", "cost": 50},
        {"id": 1, "weight": 5, "distance": 100, "priority": "high", "cost": 50},
        {"id": 2, "weight": 10, "distance": 200, "priority": "normal", "cost": 80},
        {"id": 4, "weight": 2, "distance": 50, "priority": "high", "cost": 20},
    ]


def test_pipeline_and_partial(rows: list[dict[str, object]]) -> None:
    assert pipeline(3, lambda x: x + 1, lambda x: x * 2, lambda x: x + 1) == 9
    assert pipeline(3) == 3
    operation = partial(adjust, per_kg=2, per_km=0.1, factor=1.2)
    assert operation(normalize(rows[0])).adjusted_cost == pytest.approx(84)


def test_rank_total_and_no_mutation(rows: list[dict[str, object]]) -> None:
    original = deepcopy(rows)
```

Рис. 9. Файл tests/test_lab07.py, рядки 1–33

lab07 / tests/test_lab07.py

```python
    ranked = rank_shipments(iter(rows))
    assert [s.id for s in ranked] == [1, 3, 4]
    assert total_cost(ranked) == pytest.approx(202.8)
    assert rows == original
    assert [s.id for s in rank_shipments(rows, top=1)] == [1]
    assert rank_shipments(rows, top=0) == []
    assert rank_shipments([]) == [] and total_cost([]) == 0
    assert [s.id for s in rank_shipments(rows, priority="normal")] == [2]


def test_all_sources(tmp_path: Path, rows: list[dict[str, object]]) -> None:
    text = json.dumps(rows)
    csv_text = "id,weight,distance,priority,cost\n3,5,100, HIGH
,50\n1,5,100,high,50\n2,10,200,normal,80\n4,2,50,high,20\n"
    json_file, csv_file = tmp_path / "data.json", tmp_path / "data.csv"
    json_file.write_text(text, encoding="utf-8")
    csv_file.write_text(csv_text, encoding="utf-8")
    expected = rank_shipments(rows)
    for source in (
        rows,
        text,
        json_file,
        str(json_file),
        csv_file,
        str(csv_file),
        io.StringIO(text),
        io.StringIO(csv_text),
        csv_text,
    ):
        actual = rank_shipments(load_shipments(source))
        assert [asdict(s) for s in actual] == [asdict(s) for s in expected]
    for path in (json_file, csv_file):
        with path.open(encoding="utf-8") as file:
            assert clean_and_rank(file) == expected
```

Рис. 10. Файл tests/test_lab07.py, рядки 34–66

lab07 / tests/test_lab07.py

```
67              assert not file.closed
68      loaded = load_shipments(rows)
69      loaded[0]["cost"] = 999
70      assert rows[0]["cost"] == 50
71
72
73  def test_all_comparisons_and_foreign_types(rows: list[dict[str, object]]) -> None:
74      a, b = rank_shipments(rows, top=2)
75      equivalent = normalize(
76          {"id": 1, "weight": 1, "distance": 0, "priority": "high", "cost": 84}
77      )
78      assert a == equivalent and not a != equivalent
79      assert a != b and a < b and a <= b and b > a and b >= a
80      assert a <= equivalent and a >= equivalent
81      assert sorted([b, a]) == [a, b]
82      assert a.__eq__(1) is NotImplemented
83      assert a.__lt__(1) is NotImplemented
84      assert a != 1
85      with pytest.raises(TypeError):
86          _ = a < 1
87
88
89  def test_operator_tools(rows: list[dict[str, object]]) -> None:
90      assert itemgetter("id", "weight")(rows[0]) == (3, 5)
91      assert methodcaller("strip")(" high ") == "high"
92      assert list(map(attrgetter("id"), rank_shipments(rows))) == [1, 3, 4]
93
94
95  @pytest.mark.parametrize("source", [123, (), None])
96  def test_unsupported_source(source: object) -> None:
97      with pytest.raises(TypeError):
98          load_shipments(source)
99
```

Рис. 11. Файл tests/test_lab07.py, рядки 67–99

lab07 / tests/test_lab07.py

```
100
101 @pytest.mark.parametrize("source", ["{}", "[1]", "[{}", [1]])
102 def test_invalid_payload(source: object) -> None:
103     with pytest.raises((ValueError, json.JSONDecodeError)):
104         load_shipments(source)
105
106
107 @pytest.mark.parametrize(
108     "change",
109     [
110         {"weight": 0},
111         {"cost": "nan"},
112         {"priority": "urgent"},
113         {"id": -1},
114         {"distance": -1},
115     ],
116 )
117 def test_invalid_record(
118     rows: list[dict[str, object]], change: dict[str, object]
119 ) -> None:
120     with pytest.raises(ValueError):
121         rank_shipments([{**rows[0], **change}])
122
123
124 def test_invalid_options_and_empty_csv(tmp_path: Path) -> None:
125     with pytest.raises(ValueError):
126         rank_shipments([], top=-1)
127     with pytest.raises(ValueError):
128         rank_shipments([], factor=float("inf"))
129     with pytest.raises(ValueError):
130         load_shipments(tmp_path / "x.txt")
131     assert load_shipments(io.StringIO("id,weight,distance,priority,cost\n")) == []
```

Рис. 12. Файл tests/test_lab07.py, рядки 100–131

Запуск і результати перевірки

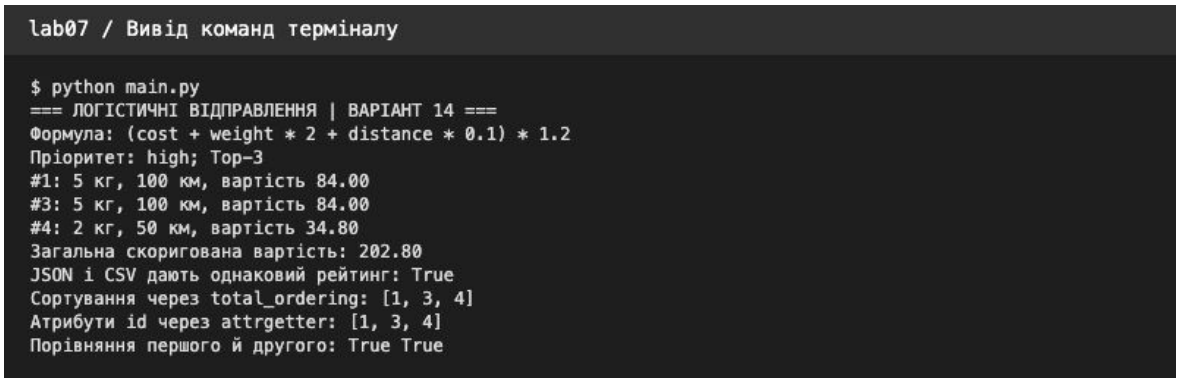


Рис. 13. Запуск демонстрації

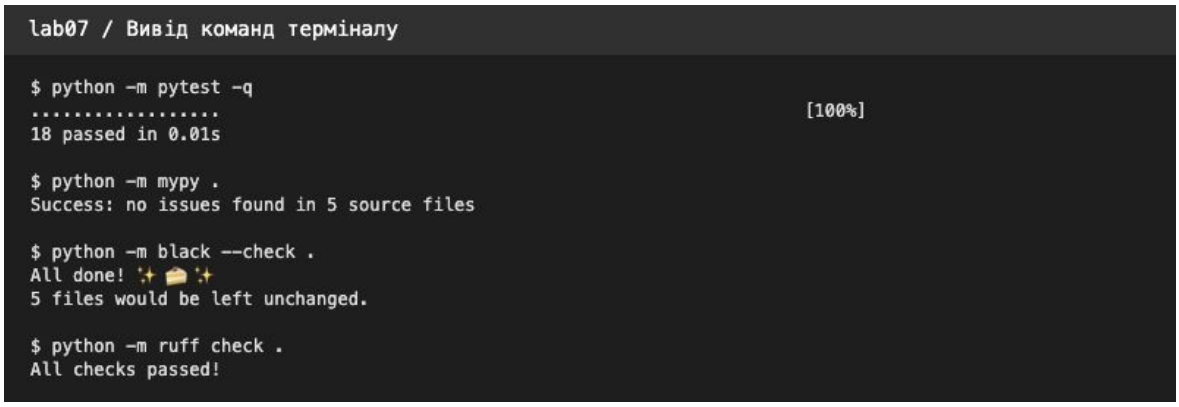


Рис. 14. Тести та перевірки якості

Висновок

Я побудував конвеєр обробки відправлень із partial, reduce та operator, додав поліморфний завантажувач JSON/CSV і модель із total_ordering. Рейтинг підтримує багатокритеріальне сортування та Top-N, початкові записи не змінюються.

Усі 18 тестів пройшли; mypy, black і ruff помилок не виявили. Різні типи джерел дають однаковий результат. Для демонстраційних тарифів сумарна скоригована вартість Top-3 становить 202,80.

Роботу виконано за наданим Lab_07.docx, індивідуальний варіант 14.